DAM
Desarrollo de Aplicaciones Multiplataforma
2º Curso

AD
Acceso a Datos

UD 2
Manejo de Ficheros
Parte 2 - JSON

IES BALMIS
Dpto Informática
Curso 2026-2027
Versión 1 (09/2026)

UD2 – Manejo de ficheros

ÍNDICE

14. Ficheros JSON
- 14.1.- Tipos de API para procesar flujos JSON
- 14.2.- Creación de un JSON a partir de datos simples
- 14.3.- Creación de un JSON desde ArrayList de Objetos
- 14.4.- Lectura de un JSON
- 14.5.- Serializar objetos con JSON

14. Ficheros JSON

Un flujo de datos en formato **JSON** se representa mediante objetos JSON, los cuales constan de una o más parejas "nombre":valor encerradas entre llaves {}. Las parejas se separan entre sí mediante comas.

El valor puede ser:

- **Simple**: Boolean, Number or String
- **Compuesto**:
  - JSONObject : Un objeto JSON con sus correspondientes {}
  - JSONArray : Un Array de valores simples o compuestos delimitados con []

JSONObject (SINTAXIS)	{ Object }
JSONArray (SINTAXIS)	[{ Object }, { Object }, { Object }]
JSONObject con valor Boolean	{ "nombre" : true }
JSONObject con valor Number	{ "nombre" : 2 }
JSONObject con valor String	{ "nombre" : "valor cadena" }
JSONObject con valor Array	{ "nombre" : [2, 7, 15] }
JSONObject VARIOS VALORES	{ "nombre1" : 2, "nombre2" : "valor cadena"}

Ejemplo de JSONObject con valores simples:

```
{
    "nombre": "Juan",
    "edad": 30,
    "encargado": false
}
```

Ejemplo de JSONObject que contiene el valor de "empleados" que es compuesto de un JSONArray de varios JSONObjects:

```
{
    "empresa": "Computer Intelligence",
    "empleados":[
        {"nombre":"John", "apellido":"Doe"},
        {"nombre":"Anna", "apellido":"Smith"},
        {"nombre":"Peter", "apellido":"Jones"}
    ]
}
```

Para ver el árbol de forma visual que genera el JSON podemos utilizar

JSON Viewer Online
http://jsonviewer.stack.hu/

14.1.- Tipos de API para procesar flujos JSON

Existen varios:

Org.json	import **org.json**
JSR (Java Specification Request) API	import **javax.json**
Gson API	import **com.google.gson.Gson**
Jackson API	import **com.fasterxml.jackson.core**

Nosotros trabajaremos principalmente con **org.json** por su simplicidad, ya que dispone de métodos similares a la clase ArrayList.

JSON Library - org.json
https://mvnrepository.com/artifact/org.json/json
JSON Library - Reference
https://javadoc.io/doc/org.json/json/latest/index.html

14.2.- Creación de un JSON a partir de datos simples

Disponemos de dos clases principales **JSONObject** y **JSONArray** dependiendo de lo que vayamos a almacenar. Crearemos el siguiente JSONObject desde Java:

```
{
    "title": "JSON-Processing With Java EE",
    "chapters": [
        "Introduction",
        "1. JSON and Java",
        "2. JSON-Processing API features",
        "3. The Java EE JSON Object model",
        "4. The Java EE JSON Streaming model",
        "Conclusion"
    ],
    "released": true,
    "length": 90,
    "sourceCode": {
        "repositoryName": "JSON-Processing-with-Java-EE",
        "url": "github.com/readlearncode"
    },
    "complementaryCourse": [
        {
            "title": "RESTful Service with JAX-RS 2.0",
            "length": 120
        },
        {
            "title": "Java Enterprise Edition Introduction",
            "length": 130
        }
    ],
    "notes": null
}
```

UD2Json01

```
JSONObject json = new JSONObject();

json.put("title", "JSON-Processing With Java EE");

JSONArray jChapters = new JSONArray();
    jChapters.put("Introduction");
    jChapters.put("1. JSON and Java");
    jChapters.put("2. JSON-Processing API features");
    jChapters.put("3. The Java EE JSON Object model");
    jChapters.put("4. The Java EE JSON Streaming model");
    jChapters.put("Conclusion");
json.put("chapters", jChapters);

json.put("released", true);
json.put("length", 90);

JSONObject jSourceCode = new JSONObject();
    jSourceCode.put("repositoryName", "JSON-Processing-with-Java-EE");
    jSourceCode.put("url", "github.com/readlearncode");
json.put("sourceCode", jSourceCode);

JSONArray jCourses = new JSONArray();
    JSONObject jElement;

    jElement = new JSONObject();
        jElement.put("title", "RESTful Service with JAX-RS 2.0");
        jElement.put("length", 120);
    jCourses.put(jElement);

    jElement = new JSONObject();
        jElement.put("title", "Java Enterprise Edition Introduction");
        jElement.put("length", 130);
    jCourses.put(jElement);

json.put("complementaryCourse",jCourses);

json.put("notes",JSONObject.NULL);


System.out.println(json.toString(2)); // Mostrar Pretty
```

14.3.- Creación de un JSON desde ArrayList de Objetos

Vamos a crear una lista de objetos en Java:

UD2Json02

```
public class Sistema implements Serializable {
  int idSist;
  String nomSist;

  public Sistema() {}

  public Sistema(int idSist, String nomSist) {
        this.idSist = idSist;
        this.nomSist = nomSist;
  }

  public int getIdSist() {
        return idSist;
  }
  public void setIdSist(int idSist) {
        this.idSist = idSist;
  }

  public String getNomSist() {
        return nomSist;
  }
  public void setNomSist(String nomSist) {
        this.nomSist = nomSist;
  }

  @Override
  public String toString() {
      return "Sistemas{" + "idSist=" + idSist +
                  ", nomSist=" + nomSist + '}';
  }
}
```

UD2Json02

```
public class UD2Json02 {
  public static void main(String[] args)  {

    ArrayList<Sistema> lista = new ArrayList<>();

    Sistema o1 = new Sistema(1,"Windows");
    Sistema o2 = new Sistema(2,"Linux");

    lista.add(o1);
    lista.add(o2);

    System.out.println(lista.toString());

  }
}
```

Ahora deseamos obtener un **JSONArray** como:

```
[
  {"idSist":1,"nomSist":"Windows"},
  {"idSist":2,"nomSist":"Linux"}
]
```

Para ello, tenemos dos opciones.

Opción 1)
En un primer paso crearemos el **JSONArray** y luego recorreremos la lista añadiendo los **JSONObjects**, uno por cada objeto de la lista:

UD2Json02

```
...
  JSONArray jArraySistemas;

  // Cargar datos mediante bucle
  jArraySistemas = new JSONArray();
  for (Sistema sis:lista) {
     JSONObject jObj = new JSONObject();
     jObj.put("idSist",sis.getIdSist());
     jObj.put("nomSist",sis.getNomSist());

     jArraySistemas.put(jObj);
  }
  System.out.println(jArraySistemas.toString(2));
...
```

Opción 2)
Usar el constructor pasándole directamente el ArrayList:

```
...
  JSONArray jArraySistemas;

  // Cargar datos mediante constructor
  jArraySistemas = new JSONArray(lista);
  System.out.println(jArraySistemas.toString(2));
...
```

Ahora para almacenarlo, usaremos por ejemplo la clase **FileWriter**:

```
...
  try (FileWriter fw = new FileWriter("./datos/sistemas.json",false)) {
     jArraySistemas.write(fw,2,0); // 2=prettyPrint, 0=tabulación
  } catch (IOException ex) {
     System.out.println("ERROR: " + ex.getMessage());
  }
...
```

14.4.- Lectura de un JSON

Para nuestro ejemplo crearemos un JSONArray con datos de empleados:

```
[
  {
    "nombre": "Alejandro Pérez",
    "edad": 26,
    "encargado": false
  },
  {
    "nombre": "Ana López",
    "edad": 28,
    "encargado": false
  },
  {
    "nombre": "Silvia Rodríguez",
    "edad": 34,
    "encargado": true
  }
]
```

Cuando hacemos un recorrido sobre un JSONArray obtendremos Object ya que puede contener cualquier tipo de datos simple o complejo.

Para realizar el recorrido generalmente usaremos **for**

```
for (Object elemento : jsonArray) {
}
```

o **foreach** (con notación Lambda)

```
jArrayEmpleados.forEach((elemento) -> {
});
```

Luego podemos realizar un "CAST" de cada elemento a lo que queramos.

En nuestro ejemplo son todos JSONObject, así que realizaremos:

```
JSONObject jObj = (JSONObject) elemento;
```

Cuando ya tenemos un **JSONObject** podemos obtener su conjunto de **Keys** y mostrar su contenido:

```
for (String key: jObj.keySet()) {
   System.out.println(key+":"+jObj.get(key));
}
```

También se puede obtener directamente el valor de un key sabiendo su tipo. En nuestro ejemplo,:

```
jObjEmpleado.getString("nombre");
jObjEmpleado.getInt("edad");
jObjEmpleado.getBoolean("encargado");
```

UD2Json03

```
// Carga de datos
JSONArray jArrayEmpleados = new JSONArray();
  JSONObject jEmpleado;

  jEmpleado = new JSONObject();
    jEmpleado.put("nombre", "Alejandro Pérez");
    jEmpleado.put("edad", 26);
    jEmpleado.put("encargado", false);
  jArrayEmpleados.put(jEmpleado);

  jEmpleado = new JSONObject();
    jEmpleado.put("nombre", "Ana López");
    jEmpleado.put("edad", 28);
    jEmpleado.put("encargado", false);
  jArrayEmpleados.put(jEmpleado);

  jEmpleado = new JSONObject();
    jEmpleado.put("nombre", "Silvia Rodríguez");
    jEmpleado.put("edad", 34);
    jEmpleado.put("encargado", true);
  jArrayEmpleados.put(jEmpleado);

System.out.println(jArrayEmpleados.toString(2));
```

UD2Json03

```
// Recorrido con for y bucle sobre Keys
System.out.println("------------------------------");
for (Object empleado: jArrayEmpleados) {
  JSONObject jObjEmpleado = (JSONObject) empleado;
  for (String key: jObjEmpleado.keySet()) {
    System.out.println(key+":"+jObjEmpleado.get(key));
  }
  System.out.println("------------------------------");
}
```

UD2Json03

```
// Recorrido con for y acceso a cada elemento
System.out.println("------------------------------");
for (Object empleado: jArrayEmpleados) {
  JSONObject jObjEmpleado = (JSONObject) empleado;
  System.out.println("NOMBRE...: " + jObjEmpleado.getString("nombre") );
  System.out.println("EDAD.....: " + jObjEmpleado.getInt("edad") );
  System.out.println("ENCARGADO: " +
                     (jObjEmpleado.getBoolean("encargado")?"SI":"NO") );
  System.out.println("------------------------------");
}
```

UD2Json03

```
  // Recorrido con for y acceso a cada elemento
  System.out.println("------------------------------");
  jArrayEmpleados.forEach((empleado) -> {
    JSONObject jObjEmpleado = (JSONObject) empleado;
    System.out.println("NOMBRE...: " + jObjEmpleado.getString("nombre") );
    System.out.println("EDAD.....: " + jObjEmpleado.getInt("edad") );
    System.out.println("ENCARGADO: " +
                       (jObjEmpleado.getBoolean("encargado")?"SI":"NO") );
    System.out.println("------------------------------");
  });
```

14.4.- Acceso a datos JSON

En el siguiente ejemplo crearemos un objeto JSON que contenga:

```
{
    "nombre": "Ana López",
    "edad": 35,
    "animales": [
        "gato",
        "perro"
    ]
}
```

Podemos comprobar que tenemos datos simples como **String** (nombre) e **Integer** (edad), pero además tenemos un **Array de Strings**.

JSONSample → creatingJsonObject

```
public static JSONObject creatingJsonObject() {
    // Array de String
    JSONArray animales = new JSONArray();
    animales.put("gato");
    animales.put("perro");

    // Object con String, Integer y Array
    JSONObject persona = new JSONObject();
    persona.put("nombre", "Ana López");
    persona.put("edad", 35);
    persona.put("animales", animales);

    return persona;
}
```

Para recuperar los datos de un **JSONObject** utilizaremos los métodos **opt** o **get** dependiendo del tipo de dato y de si queremos un valor por defecto si no existe:

JSONSample → accessingJson

```
public static JSONObject accessingJson(JSONObject json) {
    String nombre;
    int edad, age;

    try {
      nombre = json.getString("nombre");
      edad   = json.getInt("edad");
      age    = json.optInt("age", -1); // usando valor por defecto si no existe

      System.out.println("Nombre: " + nombre);
      System.out.println("Edad..: " + edad);
      System.out.println("Age...: " + age);

      System.out.println("Animales:");
      JSONArray animales = json.getJSONArray("animales");
      for (int i = 0; i < animales.length(); i++) {
          String str = animales.getString(i);
          System.out.println("--> " + str);
      }
    } catch (org.json.JSONException ex) {
      System.out.println("ERROR: " + ex.getMessage());
    }

    // Error al acceder a un campo Integer que no existe
    try {
      int number = json.getInt("name");
    } catch (org.json.JSONException ex) {
      System.out.println("ERROR: " + ex.getMessage());
    }

    // Error al acceder a un campo Object que no existe
    try {
      Object invalid = json.get("invalid");
    } catch (org.json.JSONException ex) {
      System.out.println("ERROR: " + ex.getMessage());
    }
}
```

Analizando el origen de los datos

Lo primero que debemos saber del origen en formato JSON a leer es su tipo:

- Si comienza por `{` → Es un **`JSONObject`**
- Si comienza por `[` → Es un **`JSONArray`**

Lo segundo su origen, es decir, dónde se encuentra el contenido del JSON como por ejemplo:

- String
- File
- URL

De esta forma podremos crear métodos como:

Función	Descripción
getObjectFromString(String)	Obtiene un JsonObject a partir de un String
getObjectFromFile(String)	Obtiene un JsonObject a partir del nombre de un fichero
getObjectFromURL(String)	Obtiene un JsonObject a partir de la URL

Veamos cómo podría ser el código de cada una de ellas:

```
public static JSONObject getObjectFromString(String jsonObjectStr) {
  JSONObject json;
  json = new JSONObject(jsonObjectStr);
  return json;
}
```

```
public static JSONObject getObjectFromFile(String strFile) {
   JSONObject json;
   json = new JSONObject(getStringFromFile(strFile));
   return json;
}
```

```
public static JSONObject getObjectFromUrl(String strConnection) {
   JSONObject json;
   json = new JSONObject(getStringFromUrl(strConnection));
   return json;
}
```

Como podemos comprobar, necesitaremos otros dos métodos para obtener un **String** leyendo de un fichero o de una URL

```
public static String getStringFromFile(String strFile) {

  // Lectura completa
  String jsonText = "";

  try {
     jsonText = new String(Files.readAllBytes(Paths.get(strFile)),
                           StandardCharsets.UTF_8);
  } catch (IOException ex) {
     System.out.println(ex.getMessage());
  } catch (Exception ex) {
     System.out.println(ex.getMessage());
  }
  return jsonText;
}
```

```
public static String getStringFromUrl(String strConnection) {

  String jsonText = "";

  try {
    // Conexión sin proxy
    URI uri = new URI(strConnection);
    URL url = uri.toURL();
    URLConnection uc = url.openConnection();
    HttpURLConnection conn = (HttpURLConnection) uc;

    // La conexión se va a realizar para poder enviar y recibir
    // información en formato JSON
    conn.setDoInput(true);
    conn.setDoOutput(true);
    conn.setRequestProperty("Content-type", "application/json");

    // Se va a realizar una petición con el método GET
    conn.setRequestMethod("GET");

    // Ejecutar la conexión y obtener la respuesta
    try (
          InputStreamReader rd = new InputStreamReader(conn.getInputStream());
          BufferedReader bf = new BufferedReader(rd)
        ) {

        String str;
        while ((str = bf.readLine()) != null) {
           jsonText += str.trim();
        }
     }
  } catch (MalformedURLException ex) {
     System.out.println("ERROR: " + ex.getMessage());
  } catch (IOException ex) {
     System.out.println("ERROR: " + ex.getMessage());
  }


  return jsonText;
}
```

Con proxy sería:

```
  // Conexión con proxy
  Proxy proxy = new Proxy( Proxy.Type.HTTP,
                           new InetSocketAddress("10.100.0.1", 8080));
  URI uri = new URI(strConnection);
  URL url = uri.toURL();
  URLConnection uc = url.openConnection(proxy);
  HttpURLConnection conn = (HttpURLConnection) uc;
```

De igual manera podríamos crear funciones para leer **JSONArray**, cambiando **JSONObject:**

Función	Descripción
getArrayFromString(String)	Obtiene un JsonArray a partir de un String
getArrayFromFile(String)	Obtiene un JsonArray a partir del nombre de un fichero
getArrayFromURL(String)	Obtiene un JsonArray a partir de la URL

Aprovechando las funciones anteriormente creadas, podemos leer diferentes **JSONArray** o **JSONObject** de diferentes orígenes:

```
JSONObject jObjArticulo=JSONUtils.getObjectFromString(
  "{articulo:'Teclado', precio:2.34, dto:5}"
);
```

```
JSONArray jArrayInventario=JSONUtils.getArrayFromString(
  "[ 5,'piezas', 6.34, {art:'Lámpara', color: 'plateada'}]"
);
```

```
JSONObject jObjFrutas = JSONUtils.getObjectFromFile(
  "./datos/contenedor-frutas.json"
);
```

```
JSONArray jArrayFrutas = JSONUtils.getArrayFromFile(
  "./datos/contenedores-frutas-50.json"
);
```

```
JSONObject jObjIP=JSONUtils.getObjectFromUrl(
  "https://api.myip.com/"
);
```

```
JSONArray jArrayLibros=JSONUtils.getArrayFromUrl(
  "http://riconet.es/fp/apirest/libros"
);
```

Si tenemos un **JSONArray** de varios **JSONObject**, podemos acceder a un elemento directamente a través de sus métodos:

```
System.out.println("TÍTULO LIBRO 2: " +
                   jArrayLibros.getJSONObject(1).getString("titulo"));
```

UD2Json03fromFile

Realiza la misma aplicación que en UD2Json03 pero siendo el origen un fichero.

Realizaremos el proyecto anterior siendo el origen un **JSONArray** en un archivo.

empleados.json

```
[
  {
      "nombre":"Juan",
      "edad": 30,
      "encargado": false
  },
  {
      "nombre":"Ana",
      "edad": 27,
      "encargado": true
  },
  {
      "nombre":"Elena",
      "edad": 38,
      "encargado": false
  }
]
```

La aplicación utilizará la función **getArrayFromFile** para obtener el JsonArray.

UD2Json03fromFile

```
String fichero = "./datos/empleados.json";

// Obtener un JSONArray a partir del String
JSONArray jArrayEmpleados = JSONUtils.getArrayFromFile(fichero);


// **********************************
// Imprimir contenido de JSONArray
// **********************************
System.out.println("+------------+------+-----------+");
System.out.println("| NOMBRE     | EDAD | ENCARGADO |");
System.out.println("+------------+------+-----------+");
String encargado="";
JSONObject emp;
for (Object elemento : jArrayEmpleados) {
    emp = (JSONObject) elemento;
    if (emp .getBoolean("encargado")) {
        encargado="SI";
    } else {
        encargado="NO";
    }
    System.out.printf("| %-10s | %4d | %-9s |\n",
                  emp.getString("nombre"), emp.getInt("edad"), encargado);
}
System.out.println("+------------+------+-----------+");
```

14.5.- Serializar objetos con JSON

Las librerías de **org.json** no disponen de un método automático para serializar objetos.

Si deseamos "**serializar / desserializar**" podremos utilizar las librerías de **Gson**.

Por ejemplo, si disponemos de la clase **Users**:

Users

```
public class Users implements Serializable {
    int userId;
    String userName;

    // Constructores
    public Users() {}

    public Users(int userId, String userName) {
        this.userId = userId;
        this.userName = userName;
    }

    // getters y setters
    public int getUserId() {
        return userId;
    }

    public void setUserId(int userId) {
        this.userId = userId;
    }

    public String getUserName() {
        return userName;
    }

    public void setUserName(String userName) {
        this.userName = userName;
    }

    // Sobrecarga del método toString para verlo como un JSON
    @Override
    public String toString() {
        String strObjeto;

        strObjeto = "{\"userid\":" + userId +
                                ",\"userName\":\"" + userName+ "\"}";
        return strObjeto;
    }
}
```

Ahora realizaremos una aplicación que:

1) Crear un objeto de la clase Users con los valores (1, "Juan")
2) Utilizando el método **toJson()** de la librería **Gson**, serializar los datos del objeto anterior a un String y mostrarlo por pantalla
3) A partir de un String con el contenido {'userId':2, 'userName':'Sergio'} obtener un objeto de la clase Users y mostrarlo utilizando su método toString()
4) Crear un ArraList de objetos Users y añadir dos objetos (3,"Ana") y (4,"María"). A continuación, serializa el ArrayList en un String con toJson y muéstralo por pantalla.

UD2Json04SerializarGson

```
import clases.Users;

import com.google.gson.Gson;
import com.google.gson.GsonBuilder;
import java.util.ArrayList;


public class UD2Json04SerializarGson {

    public static void main(String[] args) {
        // Crear el GsonBuilder
        Gson gson = new GsonBuilder().setPrettyPrinting().create();

        // Crear objeto y Serializarlo (Objeto a String)
        Users userData = new Users(1,"Juan");
        String userDataString = gson.toJson(userData);
        System.out.println(userDataString);

        // Deserializar objeto desde un String (String a Objeto)
        String userJson = "{'userId':2, 'userName':'Sergio'}";
        Users userObject = gson.fromJson(userJson, Users.class);
        System.out.println(userObject.toString());

        // Serializar una lista de objetos utilizando la combinación de:
            // gson.toJson que usa a su vez User.toString()
        ArrayList<Users> lista = new ArrayList<>();
        lista.add(new Users(3,"Ana"));
        lista.add(new Users(4,"María"));

        String userListaString = gson.toJson(lista);
        System.out.println(userListaString);

    }

}
```

De igual forma podríamos realizar el proceso contrario, es decir, serialización de JSON a objeto de una clase, utilizando el método **fromJson()**.

UD2Json05fromFileGson

```
Gson gson = new Gson();
String jsonString = "";

// ************************************************
// Leer contenido del fichero (De Fichero a String)
// ************************************************
File file = new File("./datos/persona.json");
try (
   FileInputStream fis = new FileInputStream(file);
   InputStreamReader isr = new InputStreamReader(fis, StandardCharsets.UTF_8);
   BufferedReader reader = new BufferedReader(isr);
  ) {

  String str;
  while ((str = reader.readLine()) != null) {
    jsonString += str.trim();
  }
} catch (FileNotFoundException ex) {
  Logger.getLogger(UD2Json05fromFileGson.class.getName())
                                                .log(Level.SEVERE, null, ex);
} catch (IOException ex) {
  Logger.getLogger(UD2Json05fromFileGson.class.getName())
                                                .log(Level.SEVERE, null, ex);
}

// ***************
// Imprimir String
// ***************
System.out.println("Contenido JSON del fichero:");
System.out.println("---------------------------");
System.out.println(jsonString);

// ******************************
// Convertir (De JSONString a RAM)
// ******************************
Persona persona = gson.fromJson(jsonString, Persona.class);

// ***********************************
// Imprimir contenido de Objeto de RAM
// ***********************************
System.out.println(persona.toString());
```

Si el fichero contiene un JsonArray en vez de un JsonObject deberemos cargar en un ArrayList y para mapearlo usaremos **TypeToken**.

UD2Json06fromFileGson

```
Gson gson = new Gson();
String jsonString = "";

// ***********************************************
// Leer contenido del fichero (De Fichero a String)
// ***********************************************
File file = new File("./datos/persona.json");
try (
   FileInputStream fis = new FileInputStream(file);
   InputStreamReader isr = new InputStreamReader(fis, StandardCharsets.UTF_8);
   BufferedReader reader = new BufferedReader(isr);
  ) {

  String str;
  while ((str = reader.readLine()) != null) {
    jsonString += str.trim();
  }
} catch (FileNotFoundException ex) {
  Logger.getLogger(UD2Json05fromFileGson.class.getName())
                                                .log(Level.SEVERE, null, ex);
} catch (IOException ex) {
  Logger.getLogger(UD2Json05fromFileGson.class.getName())
                                                .log(Level.SEVERE, null, ex);
}

// ***************
// Imprimir String
// ***************
System.out.println("Contenido JSON del fichero:");
System.out.println("---------------------------");
System.out.println(jsonString);

// ******************************
// Convertir (De JSONString a RAM)
// ******************************
ArrayList<Persona> listaPersonas
               = gson.fromJson(
                       jsonString,
                       new TypeToken<ArrayList<Persona>>(){}.getType()
                   );

// ***********************************
// Imprimir contenido de Objeto de RAM
// ***********************************
System.out.println("+-----------+------+----------+");
System.out.println("| NOMBRE    | EDAD | ENCARGADO |");
System.out.println("+-----------+------+----------+");
String encargado="";
for (Persona per : listaPersonas) {
    if (per.isEncargado()) {
        encargado="SI";
    } else {
        encargado="NO";
    }
    System.out.printf("| %-10s | %4d | %-9s |\n",
                      per.getNombre(), per.getEdad(), encargado);
}
System.out.println("+-----------+------+----------+");
```